

Experiment 01

Title: Array

Theory:

Array is defined as an ordered set of similar data items. All the data items of an array are stored in consecutive memory locations in RAM. The elements of an array are of same data type and each item can be accessed using the same name.

Representation of Array

```
int arr[10];
```

```
char arr[10];
```

```
float arr[5];
```

Now in order to work around with the array you need to access the elements inside it, to access it we use:

```
#include <stdio.h>
int main(){
    //declaring the array of type int and size 2
    int a[2]= {1,5};

    //declaring the array of type char and size 3
    char b[3]= {'a', 'b', 'c'};

    //declaring the array of type float and size 5
    float c[5] = {0.2, 0.7, 0.8, 0.21, 0.9};

    //accessing the first element of array a
    printf ("%d", a[0]);

    //accessing the elements of array b
    for (int i=0; i<2;i++){
        printf("%c\n", b[i]);
    }
    printf("%f", c[4]);
}
```

Types of Array in Data Structure

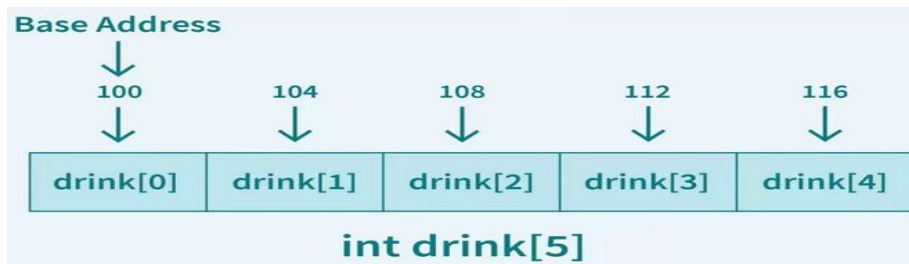
- One Dimensional Arrays

The arrays we discussed and declared in the previous section were 1D arrays because they stored elements linearly in continuous memory locations. Single dimension is used to store elements inside this array. They are denoted as Array_Name [].

- Multi Dimensional Arrays

Arrays having more than one dimension are multiple dimensional arrays as the name suggests, they are further divided into two-dimensional (2D) arrays and three-dimensional (3D) arrays.

- Two Dimensional Arrays– 2D arrays give us a tabular representation by storing elements in a form of rows(i) * columns(j), for instance an A[2][3] will have 2 rows and 3 columns allocating 6 elements. The array starts from A[0][0] which gives us the first element like shown in the image below.
- Three Dimensional Arrays – A 3D array extends a 2D array by adding a dimension depth in this data structure, so this array has depth, rows and columns denoted as A[k][i][j] where k, i, j represents depth, rows and columns respectively. So evidently A[3][3][4] will have depth 3, 3 rows and 4 column:



Operations Performed with an Array

- Traverse: Traversing literally means travelling through an area, and in terms of a programming language it means running a loop in an array and performing the required operation.
- Insertion: Adding an element inside an array on the given position(index) in the array.
- Deletion: Removing any element from an array at a given position(index).
- Search: Searches an element of an array by the element index or the value.
- Update: Updates an element on a given position(index).

Advantages of Array

- Easier to remember a group name than the individual names likewise it's easier to remember an array name than remembering names of several variables.
- Traversing which we read above is a brilliant feature of arrays, it's easy, incrementing the base address of the array lets us visit every element inside an array one after another.
- Any element of the array can be accessed by using the index(position) of the element in O(1) time complexity.

Conclusion: Thus, we have studied array data structure and operations.

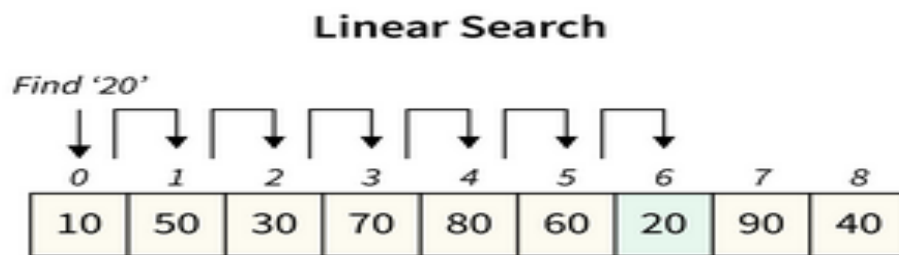
Experiment 02

Title: Searching in Array- Linear Search and Binary Search

Theory:

Searching is a method to find some relevant information in a data set. In computer programming, searching is usually referred to as finding a particular element in a data structure. Linear search algorithm is a straightforward search algorithm. This type of search performs a sequential search on all items individually.

- Linear search:



Linear search is an algorithm which is used to find a target value within an array of elements. Here's how it works:

1. Start from the beginning of the array ($i = 0$) and traverse each element sequentially.
2. If the current element equals the target value ($\text{arr}[i] == K$), return the index i .
3. If the end of the array is reached without finding the target value ($i \geq n$), return -1 to indicate that the value was not found.
4. If the current element doesn't match with the target value, move to the next element ($i++$) and repeat the process.

Linear search is a simple search-based algorithm that sequentially scans through all elements in a data collection to find a target element.

In linear search, we compare each element with the target from the start to the end of an array with time complexity of $O(n)$.

Advantages of linear search include simplicity, applicability to unsorted data, no preprocessing required, and ease of understanding.

Drawbacks of linear search include inefficiency for large data sets, unnecessary comparisons, and dependence on data distribution.

- Binary Search:

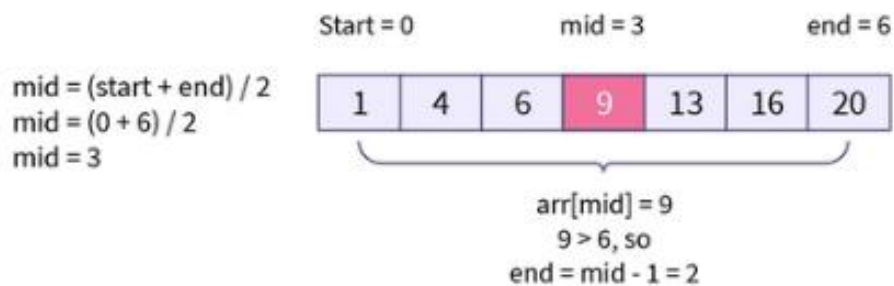
Binary Search in C is generally used to solve a wide range of issues in computer science and real-world scenarios related to searching.

One of the most efficient is the Binary Search algorithm.

Binary search algorithm operates on a sorted array to find a target element and it is more efficient than the linear search algorithm.

Binary Search Algorithm

1. Initialization:
 - Set left to the index of the first element in the search space.
 - Set right to the index of the last element in the search space.
2. Search:
 - Calculate the middle index mid as $(\text{left} + \text{right}) / 2$.
 - Compare the element at index mid with the target key.
 - If the key is found at mid, the search is successful, and the process terminates.
 - If the key is not found at mid, proceed to the next step.
3. Choose Search Space:
 - If the element at index mid is smaller than the target key, narrow down the search space to the right half by setting left to mid + 1.
 - If the element at index mid is larger than the target key, narrow down the search space to the left half by setting right to mid - 1.
4. Repeat:
 - Repeat steps 2 and 3 while left is less than or equal to right.
 - If the key is found, return mid.
5. Termination:
 - If left becomes greater than right, then the target key is not in the dataset, and the search process concludes.



- Binary Search is efficient because it continually divides the search space in half, until it finds the element or only one element remains in the list to be tested.
- Binary Search has a time complexity of **O(1)** in the best case, and **O(log N)** in the worst case.

Conclusion: Thus, we have studied Searching techniques of Array.

Experiment 03

Title: Sorting on Array- Bubble Sort, Selection Sort and Insertion Sort.

Theory:

Sorting is the act of arranging data in ascending or descending order based on linear relationships among data items. Sorting is becoming more and more important as data search can be optimized to an extremely high degree if data is stored in a sorted manner. Data can also be arranged in a more readable manner when sorted.

Bubble Sort

This is the simplest and easiest sorting algorithm. In this method, two adjacent elements are compared and swapped until they are no longer in the intended order (ie) in ascending or descending order.

Consider that we want to sort a list in ascending order, here are the steps that the algorithm would follow:

1. Start with the first element.
2. Compare the current element with the next element.
3. If the current element is greater than the next element, then swap both the elements. If not, move to the next element.
4. Repeat steps 1 – 3 until we get the sorted list.

Optimized Bubble Sort Algorithm

In the above algorithm, the loop would still run to compare all the elements. It might cause complex issues like longer execution times.

To tackle this, we can do the following:

- Create a flag variable, called swapped.
- The value of the swap is set to true if, during any iteration, swapping was done.
- Else, the value of the swap is set to false.
- After an iteration, if the value of swapped is found to be false, it means the array is sorted, and no more comparisons are required.
- We then stop the execution.

It will help in reducing the number of comparisons, and hence the execution time of the algorithm.

In other words, if a bubble sort is being performed to sort the input in ascending order, it will first compare the first two elements of the array. When the second element is smaller than the first, the algorithm will swap them, and then move on to the next.

Selection Sort

The Selection Sort is an in-place comparison-based algorithm that divides the list into two portions: the sorted portion on the left and the unsorted portion on the right. The leftmost element of the unsorted array is swapped with the smallest element from the unsorted array,

and the smallest element is part of the sorted array. The unsorted array boundary is moved one element to the right throughout this process.

The algorithm of selection sort with the following steps –

1. Run a loop from $i = 0$ till the size of the list. Under the loop –
 - Declare a variable called minIndex. minIndex helps in tracking the index of the next element that will be swapped with the element at the index i .
 - Run a nested for loop from $j = i + 1$ till the size of the list. We initialize j with $i + 1$ since it allows us to reduce the number of total comparisons made. Under this loop –
 - Check if the element at the index j is smaller than the element at the index minIndex. If it is, set minIndex equal to j . It helps us in searching for the smallest element in the unsorted sublist.
 - Swap the element at the index i with the element at index minIndex. It allows us to place the smallest element from the unsorted sublist at the end of the sorted sublist. Note that we are updating the value of minIndex each time we find an element smaller than it.

Selection sort is efficient at checking if all elements are already sorted. This means it can quickly confirm a sorted list. Selection sort is easy to understand and implement. Its simplicity makes it a good choice for small datasets.

Insertion Sort

A simple sorting algorithm, the Insertion Sort works the same as when you are sorting cards in your hand. The first card in the card game is assumed to be sorted, and then we select an unsorted card. Unsorted cards that are greater than the first will be placed on the right; otherwise, they will be placed on the left. In the same way, all cards that are unsorted are taken out and placed exactly where they belong.

Let's explore how the Insertion Sort algorithm works using the array { 13, 12, 18, 2, 5 }:

Initial Array:

- [13, 12, 18, 2, 5]

First Pass:

- Compare the first two elements (13 and 12). Since 13 is greater than 12, they are swapped to maintain ascending order.
- New array: [12, 13, 18, 2, 5]

Second Pass:

- Move to the next element (18) and compare it with its predecessors. It's already in the correct position relative to 12 and 13, so no swaps are needed.
- Array remains: [12, 13, 18, 2, 5]

Third Pass:

The focus shifts to the fourth element (2), which is compared with its predecessors to find its rightful spot.

- Initial comparison with 18 shows 2 is smaller, swap 2 and 18: [12, 13, 2, 18, 5]
- Continuing the comparison, 2 is also smaller than 13, leading to another swap: [12, 2, 13, 18, 5]
- Finally, 2 is compared with 12 and, being smaller, is swapped again, positioning it correctly at the array's start: [2, 12, 13, 18, 5] Now, 2 is in its correct position.

Fourth Pass:

Attention now turns to the last element (5).

- Initially, 5 is compared with 18 and found to be smaller, swap 18 and 5: [2, 12, 13, 5, 18]
- The comparison continues with 13, and since 5 is smaller, another swap occurs: [2, 12, 5, 13, 18]
- Lastly, 5 is compared with 12, and being smaller, necessitates the final swap, placing it in its proper position: [2, 5, 12, 13, 18]
- This final pass ensures that 5 is inserted between 2 and 12, completing the sorting process with the array now fully sorted.

Finally, the array is sorted completely.

Conclusion: Thus, we have studied Sorting techniques on Array.

Experiment 04

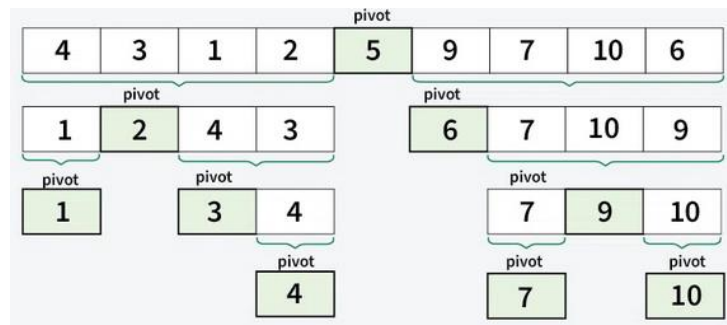
Title: Sorting on Array- Quick sort and merge sort

Theory:

Quick sort

Quicksort is a sorting algorithm based on the divide and conquer approach where

1. An array is divided into subarrays by selecting a pivot element (element selected from the array). While dividing the array, the pivot element should be positioned in such a way that elements less than pivot are kept on the left side and elements greater than pivot are on the right side of the pivot.
2. The left and right subarrays are also divided using the same approach. This process continues until each subarray contains a single element.
3. At this point, elements are already sorted. Finally, elements are combined to form a sorted array.



Choice of Pivot: Always pick the first (or last) element as a pivot.

Partition Algorithm:

```
int partition(int arr[],int low, int high) //find the partition position
{
    int pivot, start, end;
    pivot=arr[low];
    start=low;
    end=high;

    while(start<end)
    {
        while(arr[start]<=pivot)
            start++;
        while(arr[end]>pivot)
            end--;
        if(start<end)
            swap(&arr[start],&arr[end]);
    }
    swap(&arr[low], &arr[end]);
}
```

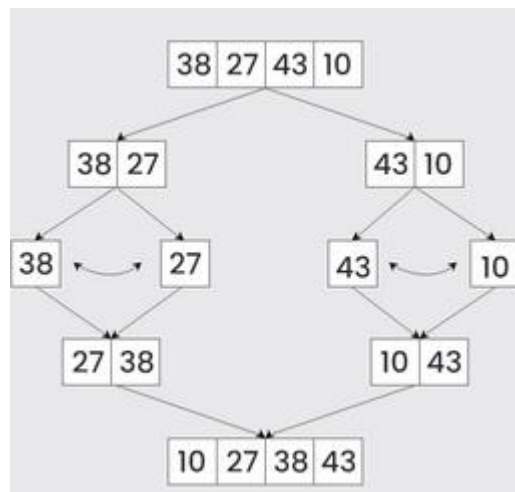
```

    return end;
}
void quick_sort(int arr[],int low, int high)
{ // find the pivot element such that
  // elements smaller than pivot are on left of pivot
  // elements greater than pivot are on right of pivot
  int pivot_loc, i;
  if(low<high)
  {
    pivot_loc=partition(arr,low,high); // Partition loc for pivot
    quick_sort(arr, low, pivot_loc-1); // recursive call on the left of pivot
    quick_sort(arr, pivot_loc+1, high); // recursive call on the right of pivot
  }
}
void swap(int *x, int *y)
{
  int t;
  t=*x;
  *x=*y;
  *y=t;
}

```

Merge sort

Merge sort works similarly works based on the divide and conquers algorithm and it is similar to quick sort. The merge sort algorithm is very efficient and popular. In this function, the list is divided into two equal halves, a call to itself is made for each half, and the two sorted halves are merged. To merge the data, we must define the merge() function.



There are multiple sub-lists that are divided into halves until the sub-lists cannot be further divided. We combine the pair of lists of one element into two-element lists, sorting them along the way. In the same way, the sorted pair of two elements gets combined with the sorted list of four elements, and so on.

MergeSort(A, p, r):

```

  if p > r
    return
  q = (p+r)/2

```

```
mergeSort(A, p, q)
mergeSort(A, q+1, r)
merge(A, p, q, r)
```

1. Divide: Divide the list or array recursively into two halves until it can no more be divided.
2. Conquer: Each subarray is sorted individually using the merge sort algorithm.
3. Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

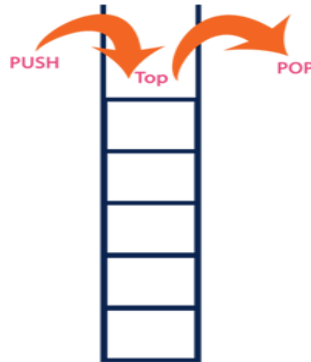
Conclusion: Thus, we have studied Sorting techniques on Array.

Experiment 05

Title: Stack using array

Theory:

Stack is a linear data structure in which the insertion and deletion operations are performed at only one end. In a stack, adding and removing of elements are performed at single position which is known as "top". That means, new element is added at top of the stack and an element is removed from the top of the stack. In stack, the insertion and deletion operations are performed based on LIFO (Last In First Out) principle.



In a stack, the insertion operation is performed using a function called "push" and deletion operation is performed using a function called "pop".

In the figure, PUSH and POP operations are performed at top position in the stack. That means, both the insertion and deletion operations are performed at one end (i.e., at Top)

Operations on a Stack

The following operations are performed on the stack...

1. Push (To insert an element on to the stack)
2. Pop (To delete an element from the stack)
3. Display (To display elements of the stack)

Stack data structure can be implement in two ways. They are as follows...

1. Using Array
2. Using Linked List

When stack is implemented using array, that stack can organize only limited number of elements. When stack is implemented using linked list, that stack can organize unlimited number of elements.

Stack Using Array

A stack data structure can be implemented using one dimensional array. But stack implemented using array, can store only fixed number of data values. This implementation is very simple, just define a one dimensional array of specific size and insert or delete the values into that array

by using LIFO principle with the help of a variable 'top'. Initially top is set to -1. Whenever we want to insert a value into the stack, increment the top value by one and then insert. Whenever we want to delete a value from the stack, then delete the top value and decrement the top value by one.

Stack Operations using Array-

Before implementing actual operations, first follow the below steps to create an empty stack.

Step 1: Include all the header files which are used in the program and define a constant 'SIZE' with specific value.

Step 2: Declare all the functions used in stack implementation.

Step 3: Create a one dimensional array with fixed size (int stack[SIZE])

Step 4: Define integer variable 'top' and initialize with '-1'. (int top = -1)

Step 5: In main method display menu with list of operations and make suitable function calls to perform operation selected by the user on the stack.

push(value) - Inserting value into the stack

In a stack, push() is a function used to insert an element into the stack. In a stack, the new element is always inserted at **top** position. Push function takes one integer value as parameter and inserts that value into the stack. We can use the following steps to push an element on to the stack...

Step 1: Check whether stack is FULL. (top == SIZE-1)

Step 2: If it is FULL, then display "Stack is FULL!!! Insertion is not possible!!!" and terminate the function.

Step 3: If it is NOT FULL, then increment top value by one (top++) and set stack[top] to value (stack[top] = value).

pop() - Delete a value from the Stack

In a stack, pop() is a function used to delete an element from the stack. In a stack, the element is always deleted from **top** position. Pop function does not take any value as parameter. We can use the following steps to pop an element from the stack...

Step 1: Check whether stack is EMPTY. (top == -1)

Step 2: If it is EMPTY, then display "Stack is EMPTY!!! Deletion is not possible!!!" and terminate the function.

Step 3: If it is NOT EMPTY, then delete stack[top] and decrement top value by one (top--).

display() - Displays the elements of a Stack

We can use the following steps to display the elements of a stack...

Step 1: Check whether stack is EMPTY. (top == -1)

Step 2: If it is EMPTY, then display "Stack is EMPTY!!!" and terminate the function.

Step 3: If it is NOT EMPTY, then define a variable 'i' and initialize with top.

Display stack[i] value and decrement i value by one (i--).

Step 3: Repeat above step until i value becomes '0'.

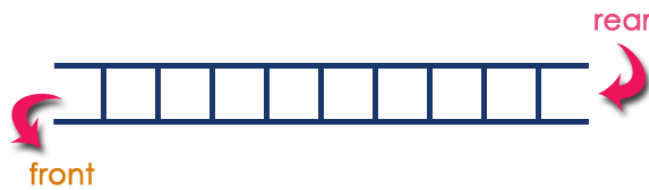
Conclusion: Thus, we have studied Stack using array.

Experiment 06

Title: Queue using array-linear queue and circular queue

Theory:

Queue is a linear data structure in which the insertion and deletion operations are performed at two different ends. In a queue data structure, adding and removing of elements are performed at two different positions. The insertion is performed at one end and deletion is performed at other end. In a queue data structure, the insertion operation is performed at a position which is known as 'rear' and the deletion operation is performed at a position which is known as 'front'. In queue data structure, the insertion and deletion operations are performed based on FIFO (First In First Out) principle.



Operations on a Queue-

The following operations are performed on a queue data structure...

1. enqueue(value) - (To insert an element into the queue)
2. dequeue() - (To delete an element from the queue)
3. display() - (To display the elements of the queue)

Queue data structure can be implemented in two ways. They are as follows...

1. Using Array
2. Using Linked List

When a queue is implemented using array, that queue can organize only limited number of elements. When a queue is implemented using linked list, that queue can organize unlimited number of elements.

Before we implement actual operations, first follow the below steps to create an empty queue.

Step 1: Include all the header files which are used in the program and define a constant 'SIZE' with specific value.

Step 2: Declare all the user defined functions which are used in queue implementation.

Step 3: Create a one dimensional array with above defined SIZE (int queue[SIZE])

Step 4: Define two integer variables 'front' and 'rear' and initialize both with '-1'. (int front = -1, rear = -1)

Step 5: Then implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on queue.

enqueue(value) - Inserting value into the queue

- In a queue data structure, enqueue() is a function used to insert a new element into the queue. In a queue, the new element is always inserted at rear position. The enqueue() function takes one integer value as parameter and inserts that value into the queue. We can use the following steps to insert an element into the queue...
 - Step 1: Check whether queue is FULL. (rear == SIZE-1)
 - Step 2: If it is FULL, then display "Queue is FULL!!! Insertion is not possible!!!" and terminate the function.
 - Step 3: If it is NOT FULL, then increment rear value by one (rear++) and set queue[rear] = value.

dequeue() - Deleting a value from the Queue

In a queue data structure, dequeue() is a function used to delete an element from the queue. In a queue, the element is always deleted from front position. The dequeue() function does not take any value as parameter. We can use the following steps to delete an element from the queue...

Step 1: Check whether queue is EMPTY. (front == rear)

Step 2: If it is EMPTY, then display "Queue is EMPTY!!! Deletion is not possible!!!" and terminate the function.

Step 3: If it is NOT EMPTY, then increment the front value by one (front ++). Then display queue[front] as deleted element. Then check whether both front and rear are equal (front == rear), if it TRUE, then set both front and rear to '-1' (front = rear = -1).

display() - Displays the elements of a Queue

We can use the following steps to display the elements of a queue...

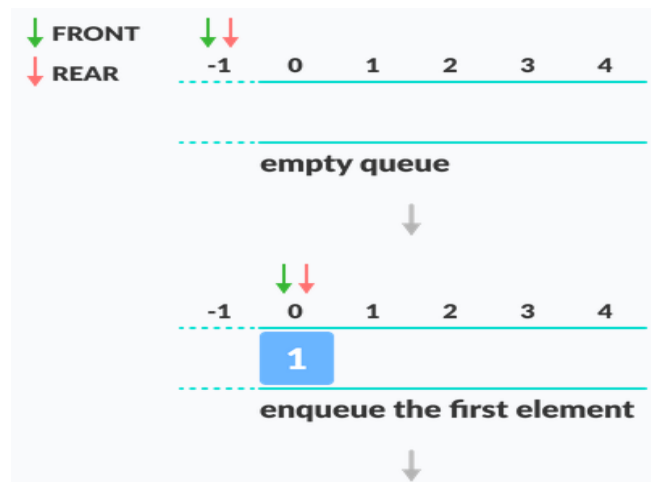
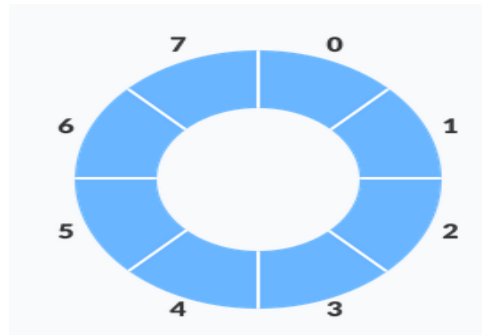
Step 1: Check whether queue is EMPTY. (front == rear)

Step 2: If it is EMPTY, then display "Queue is EMPTY!!!" and terminate the function.

Step 3: If it is NOT EMPTY, then define an integer variable 'i' and set 'i = front+1'.

Step 4: Display 'queue[i]' value and increment 'i' value by one (i++). Repeat the same until 'i' value is equal to rear (i <= rear)

A circular queue is the extended version of a regular queue where the last element is connected to the first element. Thus forming a circle-like structure.



Circular Queue works by the process of circular increment i.e. when we try to increment the pointer and we reach the end of the queue, we start from the beginning of the queue.

Here, the circular increment is performed by modulo division with the queue size.

Circular Queue Operations

The circular queue work as follows:

- two pointers *FRONT* and *REAR*
- *FRONT* track the first element of the queue
- *REAR* track the last elements of the queue
- initially, set value of *FRONT* and *REAR* to -1

1. Enqueue Operation

- check if the queue is full
- for the first element, set value of *FRONT* to 0
- circularly increase the *REAR* index by 1 (i.e. if the rear reaches the end, next it would be at the start of the queue)
- add the new element in the position pointed to by *REAR*

2. Dequeue Operation

- check if the queue is empty
- return the value pointed by *FRONT*

- circularly increase the *FRONT* index by 1
- for the last element, reset the values of *FRONT* and *REAR* to -1

However, the check for full queue has a new additional case:

- Case 1: $FRONT = 0 \ \&\& \ REAR == SIZE - 1$
- Case 2: $FRONT = REAR + 1$

Conclusion: Thus, we have studied Queue using array.

Experiment 07

Title: Linked List-Single LL, Double LL and Circular LL

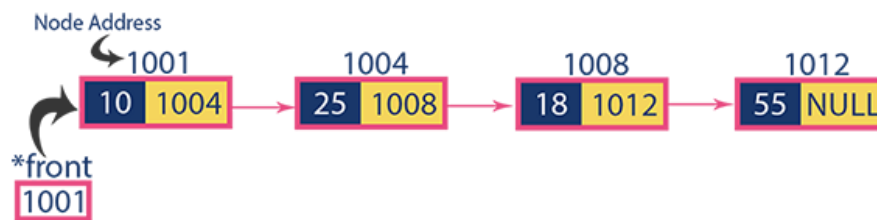
Theory:

Linked list is a linear data structure that contains sequence of elements such that each element links to its next element in the sequence. Each element in a linked list is called as "Node".

Single linked list is a sequence of elements in which every element has link to its next element in the sequence.



Ex.



Operations

In a single linked list we perform the following operations...

1. Insertion
2. Deletion
3. Display

Before we implement actual operations, first we need to setup empty list. First perform the following steps before implementing actual operations.

Step 1: Include all the header files which are used in the program.

Step 2: Declare all the user defined functions.

Step 3: Define a Node structure with two members data and next

Step 4: Define a Node pointer 'head' and set it to NULL.

Step 5: Implement the main method by displaying operations menu and make suitable function calls in the main method to perform user selected operation.

Insertion

In a single linked list, the insertion operation can be performed in three ways. They are as follows...

1. Inserting At Beginning of the list

2. Inserting At End of the list
3. Inserting At Specific location in the list

Inserting At Beginning of the list-

We can use the following steps to insert a new node at beginning of the single linked list...

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, set newNode→next = NULL and head = newNode.

Step 4: If it is Not Empty then, set newNode→next = head and head = newNode.

Inserting At End of the list-

We can use the following steps to insert a new node at end of the single linked list...

Step 1: Create a newNode with given value and newNode → next as NULL.

Step 2: Check whether list is Empty (head == NULL).

Step 3: If it is Empty then, set head = newNode.

Step 4: If it is Not Empty then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the last node in the list (until temp → next is equal to NULL).

Step 6: Set temp → next = newNode.

Inserting At Specific location in the list (After a Node)-

We can use the following steps to insert a new node after a node in the single linked list...

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, set newNode → next = NULL and head = newNode.

Step 4: If it is Not Empty then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the node after which we want to insert the newNode (until temp1 → data is equal to location, here location is the node value after which we want to insert the newNode).

Step 6: Every time check whether temp is reached to last node or not. If it is reached to last node then display 'Given node is not found in the list!!! Insertion not possible!!!' and terminate the function. Otherwise move the temp to next node.

Step 7: Finally, Set 'newNode → next = temp → next' and 'temp → next = newNode'

Deletion-

In a single linked list, the deletion operation can be performed in three ways. They are as follows...

1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list-

We can use the following steps to delete a node from beginning of the single linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4: Check whether list is having only one node ($\text{temp} \rightarrow \text{next} == \text{NULL}$)

Step 5: If it is TRUE then set $\text{head} = \text{NULL}$ and delete temp (Setting Empty list conditions)

Step 6: If it is FALSE then set $\text{head} = \text{temp} \rightarrow \text{next}$, and delete temp.

Deleting from End of the list-

We can use the following steps to delete a node from end of the single linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.

Step 4: Check whether list has only one Node ($\text{temp1} \rightarrow \text{next} == \text{NULL}$)

Step 5: If it is TRUE. Then, set $\text{head} = \text{NULL}$ and delete temp1. And terminate the function. (Setting Empty list condition)

Step 6: If it is FALSE. Then, set ' $\text{temp2} = \text{temp1}$ ' and move temp1 to its next node. Repeat the same until it reaches to the last node in the list. (until $\text{temp1} \rightarrow \text{next} == \text{NULL}$)

Step 7: Finally, Set $\text{temp2} \rightarrow \text{next} = \text{NULL}$ and delete temp1.

Deleting a Specific Node from the list-

We can use the following steps to delete a specific node from the single linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.

Step 4: Keep moving the temp1 until it reaches to the exact node to be deleted or to the last node. And every time set 'temp2 = temp1' before moving the 'temp1' to its next node.

Step 5: If it is reached to the last node then display 'Given node not found in the list! Deletion not possible!!!'. And terminate the function.

Step 6: If it is reached to the exact node which we want to delete, then check whether list is having only one node or not

Step 7: If list has only one node and that is the node to be deleted, then set head = NULL and delete temp1 (free(temp1)).

Step 8: If list contains multiple nodes, then check whether temp1 is the first node in the list (temp1 == head).

Step 9: If temp1 is the first node then move the head to the next node (head = head → next) and delete temp1.

Step 10: If temp1 is not first node then check whether it is last node in the list (temp1 → next == NULL).

Step 11: If temp1 is last node then set temp2 → next = NULL and delete temp1 (free(temp1)).

Step 12: If temp1 is not first node and not last node then set temp2 → next = temp1 → next and delete temp1 (free(temp1)).

Displaying a Single Linked List-

We can use the following steps to display the elements of a single linked list...

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!!' and terminate the function.

Step 3: If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

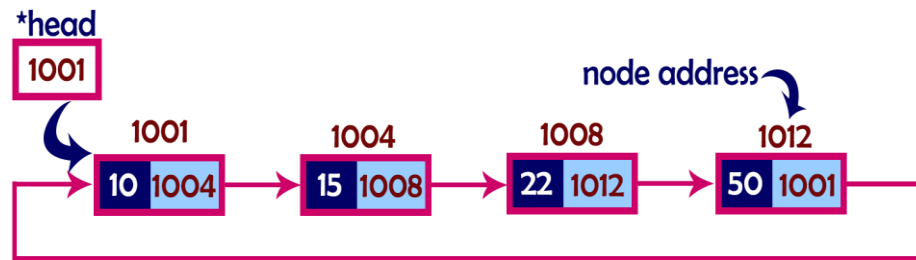
Step 4: Keep displaying temp → data with an arrow (--->) until temp reaches to the last node

Step 5: Finally display temp → data with arrow pointing to NULL (temp → data ---> NULL).

Circular linked list

Circular linked list is a sequence of elements in which every element has link to its next element in the sequence and the last element has a link to the first element in the sequence.

Example



Operations

In a circular linked list, we perform the following operations...

1. Insertion
2. Deletion
3. Display

Before we implement actual operations, first we need to setup empty list. First perform the following steps before implementing actual operations.

Step 1: Include all the header files which are used in the program.

Step 2: Declare all the user defined functions.

Step 3: Define a Node structure with two members data and next

Step 4: Define a Node pointer 'head' and set it to NULL.

Step 5: Implement the main method by displaying operations menu and make suitable function calls in the main method to perform user selected operation.

Insertion-

In a circular linked list, the insertion operation can be performed in three ways. They are as follows...

1. Inserting At Beginning of the list
2. Inserting At End of the list
3. Inserting At Specific location in the list

Inserting At Beginning of the list

We can use the following steps to insert a new node at beginning of the circular linked list...

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, set head = newNode and newNode→next = head .

Step 4: If it is Not Empty then, define a Node pointer 'temp' and initialize with 'head'.

Step 5: Keep moving the 'temp' to its next node until it reaches to the last node (until 'temp → next == head').

Step 6: Set 'newNode → next = head', 'head = newNode' and 'temp → next = head'.

Inserting At End of the list

We can use the following steps to insert a new node at end of the circular linked list...

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL).

Step 3: If it is Empty then, set head = newNode and newNode → next = head.

Step 4: If it is Not Empty then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the last node in the list (until temp → next == head).

Step 6: Set temp → next = newNode and newNode → next = head.

Inserting At Specific location in the list (After a Node)

We can use the following steps to insert a new node after a node in the circular linked list...

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, set head = newNode and newNode → next = head.

Step 4: If it is Not Empty then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the node after which we want to insert the newNode (until temp1 → data is equal to location, here location is the node value after which we want to insert the newNode).

Step 6: Every time check whether temp is reached to the last node or not. If it is reached to last node then display 'Given node is not found in the list!!! Insertion not possible!!!' and terminate the function. Otherwise move the temp to next node.

Step 7: If temp is reached to the exact node after which we want to insert the newNode then check whether it is last node (temp → next == head).

Step 8: If temp is last node then set temp → next = newNode and newNode → next = head.

Step 8: If temp is not last node then set newNode → next = temp → next and temp → next = newNode.

Deletion

In a circular linked list, the deletion operation can be performed in three ways those are as follows...

1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list

We can use the following steps to delete a node from beginning of the circular linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize both 'temp1' and 'temp2' with head.

Step 4: Check whether list is having only one node ($\text{temp1} \rightarrow \text{next} == \text{head}$)

Step 5: If it is TRUE then set $\text{head} = \text{NULL}$ and delete temp1 (Setting Empty list conditions)

Step 6: If it is FALSE move the temp1 until it reaches to the last node. (until $\text{temp1} \rightarrow \text{next} == \text{head}$)

Step 7: Then set $\text{head} = \text{temp2} \rightarrow \text{next}$, $\text{temp1} \rightarrow \text{next} = \text{head}$ and delete temp2.

Deleting from End of the list-

We can use the following steps to delete a node from end of the circular linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.

Step 4: Check whether list has only one Node ($\text{temp1} \rightarrow \text{next} == \text{head}$)

Step 5: If it is TRUE. Then, set $\text{head} = \text{NULL}$ and delete temp1. And terminate from the function. (Setting Empty list condition)

Step 6: If it is FALSE. Then, set $\text{temp2} = \text{temp1}$ and move temp1 to its next node. Repeat the same until temp1 reaches to the last node in the list. (until $\text{temp1} \rightarrow \text{next} == \text{head}$)

Step 7: Set $\text{temp2} \rightarrow \text{next} = \text{head}$ and delete temp1.

Deleting a Specific Node from the list

We can use the following steps to delete a specific node from the circular linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.

Step 4: Keep moving the temp1 until it reaches to the exact node to be deleted or to the last node. And every time set 'temp2 = temp1' before moving the 'temp1' to its next node.

Step 5: If it is reached to the last node then display 'Given node not found in the list! Deletion not possible!!!'. And terminate the function.

Step 6: If it is reached to the exact node which we want to delete, then check whether list is having only one node ($\text{temp1} \rightarrow \text{next} == \text{head}$)

Step 7: If list has only one node and that is the node to be deleted then set $\text{head} = \text{NULL}$ and delete temp1 ($\text{free}(\text{temp1})$).

Step 8: If list contains multiple nodes then check whether temp1 is the first node in the list ($\text{temp1} == \text{head}$).

Step 9: If temp1 is the first node then set $\text{temp2} = \text{head}$ and keep moving temp2 to its next node until temp2 reaches to the last node. Then set $\text{head} = \text{head} \rightarrow \text{next}$, $\text{temp2} \rightarrow \text{next} = \text{head}$ and delete temp1.

Step 10: If temp1 is not first node then check whether it is last node in the list ($\text{temp1} \rightarrow \text{next} == \text{head}$).

Step 11: If temp1 is last node then set $\text{temp2} \rightarrow \text{next} = \text{head}$ and delete temp1 ($\text{free}(\text{temp1})$).

Step 12: If temp1 is not first node and not last node then set $\text{temp2} \rightarrow \text{next} = \text{temp1} \rightarrow \text{next}$ and delete temp1 ($\text{free}(\text{temp1})$).

Displaying a circular Linked List

We can use the following steps to display the elements of a circular linked list...

Step 1: Check whether list is Empty ($\text{head} == \text{NULL}$)

Step 2: If it is Empty, then display 'List is Empty!!!' and terminate the function.

Step 3: If it is Not Empty then, define a Node pointer 'temp' and initialize with head.

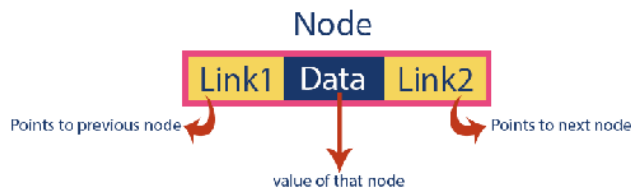
Step 4: Keep displaying $\text{temp} \rightarrow \text{data}$ with an arrow (--->) until temp reaches to the last node

Step 5: Finally display $\text{temp} \rightarrow \text{data}$ with arrow pointing to $\text{head} \rightarrow \text{data}$.

Double Linked List

Double linked list is a sequence of elements in which every element has links to its previous element and next element in the sequence.

In double linked list, every node has link to its previous node and next node. So, we can traverse forward by using next field and can traverse backward by using previous field. Every node in a double linked list contains three fields and they are shown in the following figure...



Example



Operations

In a double linked list, we perform the following operations...

1. Insertion
2. Deletion
3. Display

Insertion

In a double linked list, the insertion operation can be performed in three ways as follows...

1. Inserting At Beginning of the list
2. Inserting At End of the list
3. Inserting At Specific location in the list

Inserting At Beginning of the list

We can use the following steps to insert a new node at beginning of the double linked list...

- Step 1: Create a newNode with given value and newNode → previous as NULL.
- Step 2: Check whether list is Empty (head == NULL)
- Step 3: If it is Empty then, assign NULL to newNode → next and newNode to head.
- Step 4: If it is not Empty then, assign head to newNode → next and newNode to head.

Inserting At End of the list-

We can use the following steps to insert a new node at end of the double linked list...

- Step 1: Create a newNode with given value and newNode → next as NULL.
- Step 2: Check whether list is Empty (head == NULL)
- Step 3: If it is Empty, then assign NULL to newNode → previous and newNode to head.
- Step 4: If it is not Empty, then, define a node pointer temp and initialize with head.

Step 5: Keep moving the temp to its next node until it reaches to the last node in the list (until temp → next is equal to NULL).

Step 6: Assign newNode to temp → next and temp to newNode → previous.

Inserting At Specific location in the list (After a Node)-

We can use the following steps to insert a new node after a node in the double linked list...

Step 1: Create a newNode with given value.

Step 2: Check whether list is Empty (head == NULL)

Step 3: If it is Empty then, assign NULL to newNode → previous & newNode → next and newNode to head.

Step 4: If it is not Empty then, define two node pointers temp1 & temp2 and initialize temp1 with head.

Step 5: Keep moving the temp1 to its next node until it reaches to the node after which we want to insert the newNode (until temp1 → data is equal to location, here location is the node value after which we want to insert the newNode).

Step 6: Every time check whether temp1 is reached to the last node. If it is reached to the last node then display 'Given node is not found in the list!!! Insertion not possible!!!' and terminate the function. Otherwise move the temp1 to next node.

Step 7: Assign temp1 → next to temp2, newNode to temp1 → next, temp1 to newNode → previous, temp2 to newNode → next and newNode to temp2 → previous

Deletion-

In a double linked list, the deletion operation can be performed in three ways as follows...

1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list-

We can use the following steps to delete a node from beginning of the double linked list...

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4: Check whether list is having only one node (temp → previous is equal to temp → next)

Step 5: If it is TRUE, then set head to NULL and delete temp (Setting Empty list conditions)

Step 6: If it is FALSE, then assign temp → next to head, NULL to head → previous and delete temp.

Deleting from End of the list-

We can use the following steps to delete a node from end of the double linked list...

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty, then display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is not Empty then, define a Node pointer 'temp' and initialize with head.

Step 4: Check whether list has only one Node (temp → previous and temp → next both are NULL)

Step 5: If it is TRUE, then assign NULL to head and delete temp. And terminate from the function. (Setting Empty list condition)

Step 6: If it is FALSE, then keep moving temp until it reaches to the last node in the list. (until temp → next is equal to NULL)

Step 7: Assign NULL to temp → previous → next and delete temp.

Deleting a Specific Node from the list-

We can use the following steps to delete a specific node from the double linked list...

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminate the function.

Step 3: If it is not Empty, then define a Node pointer 'temp' and initialize with head.

Step 4: Keep moving the temp until it reaches to the exact node to be deleted or to the last node.

Step 5: If it is reached to the last node, then display 'Given node not found in the list! Deletion not possible!!!' and terminate the function.

Step 6: If it is reached to the exact node which we want to delete, then check whether list is having only one node or not

Step 7: If list has only one node and that is the node which is to be deleted then set head to NULL and delete temp (free(temp)).

Step 8: If list contains multiple nodes, then check whether temp is the first node in the list (temp == head).

Step 9: If temp is the first node, then move the head to the next node (head = head → next), set head of previous to NULL (head → previous = NULL) and delete temp.

Step 10: If temp is not the first node, then check whether it is the last node in the list (temp → next == NULL).

Step 11: If temp is the last node then set temp of previous of next to NULL (temp → previous → next = NULL) and delete temp (free(temp)).

Step 12: If temp is not the first node and not the last node, then set temp of previous of next to temp of next (temp → previous → next = temp → next), temp of next of previous to temp of previous (temp → next → previous = temp → previous) and delete temp (free(temp)).

Displaying a Double Linked List-

We can use the following steps to display the elements of a double linked list...

Step 1: Check whether list is Empty (head == NULL)

Step 2: If it is Empty, then display 'List is Empty!!!' and terminate the function.

Step 3: If it is not Empty, then define a Node pointer 'temp' and initialize with head.

Step 4: Display 'NULL <--- '.

Step 5: Keep displaying temp → data with an arrow (<===>) until temp reaches to the last node

Step 6: Finally, display temp → data with arrow pointing to NULL (temp → data ---> NULL).

Conclusion: Thus, we have studied Linked List and its types.

Experiment 08

Title: Linked List- Stack using LL and Queue using LL

Theory:

Stack using Linked List:

A stack can also be implemented using arrays. But arrays are of limited size, and the size of the stack has to be predetermined, whereas, in a linked list, implementation nodes can be added according to the user's requirements.

Node Structure:

```
struct Node {
    int data;
    struct Node *next;
};

Node* top = NULL;
```

push()

Adding or inserting a new element to a stack is known as the Push() operation in the stack.

Elements can only be pushed at the top of the stack.

Steps to push an element into a Stack:

```
if the top is equal to NULL
    newNode -> next = NULL
else
    newNode -> next = top
```

pop()

Deleting element from a stack is known as the Pop() operation in the stack. Elements can only be popped from the top of the stack.

```
if top == NULL
    print "Stack Underflow"
else
    create temporary node, *temp = top
    create temporary variable, temp_data = top->data
    top = top->next
    free(temp)

return temp_data
```

Queue using Linked List:

A linked queue consists of two pointers, i.e., the front pointer and the rear pointer. The front pointer stores the address of the first element of the queue, and the rear pointer stores the address of the last element of the queue.

Insertion is performed at the rear end, whereas deletion is performed at the front end of the queue. If front and rear both points to NULL, it signifies that the queue is empty.

The two main operations performed on the linked queue are:

- Insertion
- Deletion

Insertion

Insert operation or insertion on a linked queue adds an element to the end of the queue. The new element which is added becomes the last element of the queue.

1. Create a new node pointer.
`ptr = (struct node *) malloc (sizeof(struct node));`
2. Now, two conditions arise, i.e., either the queue is empty, or the queue contains at least one element.
3. If the queue is empty, then the new node added will be both front and rear, and the next pointer of front and rear will point to NULL.

```
*ptr -> data = val;
```

```
if (front == NULL) {
    front = ptr;
    rear = ptr;
    front -> next = NULL;
    rear -> next = NULL;
}
```

4. If the queue contains at least one element, then the condition `front == NULL` becomes false. So, make the next pointer of rear point to new node ptr and point the rear pointer to the newly created node ptr

```
rear -> next = ptr;
```

```
rear = ptr;
```

Deletion

Deletion or delete operation on a linked queue removes the element which was first inserted in the queue, i.e., always the first element of the queue is removed.

Steps to perform Deletion on a linked queue:

1. Check if the queue is empty or not.
2. If the queue is empty, i.e., `front==NULL`, so we just print 'underflow' on the screen and exit.

3. If the queue is not empty, delete the element at which the front pointer is pointing. For deleting a node, copy the node which is pointed by the front pointer into the pointer ptr and make the front pointer point to the front's next node and free the node pointed by the node ptr.

```
*ptr = front;
```

```
front = front -> next;
```

```
free(ptr);
```

Conclusion: Thus, we have studied stack and queue using Linked List.

Experiment 09

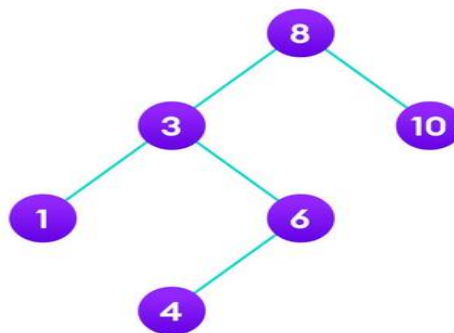
Title: Tree-Binary search tree

Theory:

A binary search tree is a tree data structure that allows the user to store elements in a sorted manner. It is called a binary tree because each node can have a maximum of two children and is called a search tree because we can search for a number in $O(\log(n))$ time.

The properties of a regular binary tree are:

1. All nodes in the left subtree have a value less than the root node
2. All nodes in the right subtree have a value more than the root node
3. Both subtrees of each node are also binary search trees, i.e. they have the above two properties



Operations to Perform on Binary Search Tree

- Search Operation-

In Search, we have to find a specific element in the data structure. This searching operation becomes simpler in binary search trees because here elements are stored in sorted order.

Algorithm for searching an element in a binary tree is as follows:

1. Compare the element to be searched with the root node of the tree.
2. If the value of the element to be searched is equal to the value of the root node, return the root node.
3. If the value does not match, check whether the value is less than the root element or not if it is then traverse the left subtree.
4. If larger than the root element, traverse the right subtree.
5. If the element is not found in the whole tree, return NULL.

Insert Operation-

Inserting an element in a binary search tree is always done at the leaf node. To perform insertion in a binary search tree, we start our search operation from the root node, if the element to be

inserted is less than the root value or the root node, then we search for an empty location in the left subtree, else, we search for the empty location in the right subtree.

```

if node == NULL
    return createNode(data)
if (data < node->data)
    node->left = insert(node->left, data)
else if (data > node->data)
    node->right = insert(node->right, data)
return node

```

Deletion Operation-

In the deletion operation, we have to delete a node from the binary search tree in a way that does not violate its properties. Deletion can occur in three possible cases:

1. Node to be deleted is the leaf node

This is the simplest case of deleting a node in a binary search tree. Here, we will replace the leaf node with NULL and free the allocated space.

2. Node to be deleted has a single child node

In this case, we will replace the target node with its child and then delete that replaced child node. This means that the child node will now contain the value to be deleted. So, we will just replace the child node with NULL and free up the allocated space.

3. The node to be deleted has two children

Here we follow certain steps to delete the node:

- We will find the inorder successor of the target node, which has to be deleted
- Now, replace this node with the successor until the target node is at the leaf
- Replace the target node with NULL and free up the allocated space

Conclusion: Thus, we have studied tree data structure.

Experiment 10

Title: Graph- Depth First Search (DFS) and Breadth first search (BFS)

Theory:

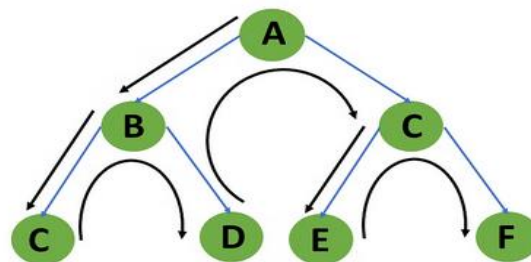
Graph traversal aims to systematically visit all vertices and edges in a graph without repeats. It includes:

- Breadth-First Search (BFS), which starts at a root node, explores all neighbors at the current depth, then proceeds to the next depth, using a queue for tracking. It's ideal for finding shortest paths in unweighted graphs.
- Depth-First Search (DFS) delves deep into the graph, exploring each branch fully before backtracking, using a stack for path tracking. Useful for puzzles, topological sorting, and identifying connected components.

Depth First Search (DFS)-

It is an algorithm that is mainly used to traverse the graph data structure. The algorithm starts from an arbitrary node (root node in case of trees) and explore as far as possible in the graph before backtracking.

After backtracking it repeats the same process for all the remaining vertices which have not been visited till now.

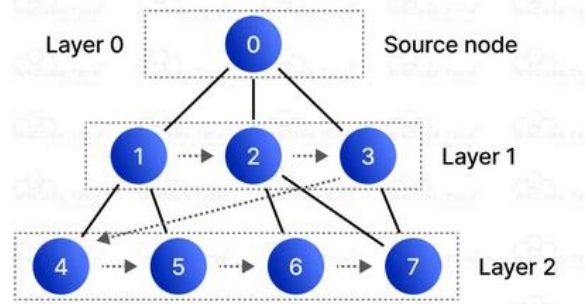


Algorithm of DFS Algorithm-

1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

Breadth First Search (BFS)

It is a fundamental graph traversal algorithm. It begins with a node, then first traverses all its adjacent nodes. Once all adjacent are visited, then their adjacent are traversed. BFS is different from DFS in a way that closest vertices are visited before others. We mainly traverse vertices level by level.



Follow the below given approach:

1. Initialization: Enqueue the given source vertex into a queue and mark it as visited.
2. Exploration: While the queue is not empty:
 - Dequeue a node from the queue and visit it (e.g., print its value).
 - For each unvisited neighbor of the dequeued node:
 - Enqueue the neighbor into the queue.
 - Mark the neighbor as visited.
3. Termination: Repeat step 2 until the queue is empty.

Conclusion: Thus, we have studied graph and its traversals.